

PRAKTIKUM MACHINE LEARNING

Bab 4

Feature Engineering: Konstruksi, Seleksi Fitur, dan Pipeline Scikit-Learn

Kumpulan Praktikum Machine Learning

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

2 Oktober 2026

Daftar Isi

Feature Engineering: Konstruksi, Seleksi Fitur, dan Pipeline Scikit-Learn	2
Ringkasan Konsep	3
4.10 - 4.14 Implementasi Python (Modul Bab 4)	4
Praktikum 4.1 - Persiapan Library (Modul Bab 4, bagian 4.10)	4
Praktikum 4.2 - Memuat Dataset California Housing (Modul Bab 4, bagian 4.10)	5
4.11 Feature Construction & Preprocessing (ColumnTransformer) (Modul Bab 4)	7
4.12 Training Pipeline (Modul Bab 4)	10
4.13 Evaluasi Model (Modul Bab 4)	11
4.14 Visualisasi Prediksi vs Aktual (Modul Bab 4)	11
4.15 Interpretasi (Modul Bab 4)	13
Eksperimen (Coba-Coba)	13
Eksperimen A - Pipeline vs Preprocessing Manual	13
Eksperimen B - Dengan vs Tanpa Feature Construction	15
Eksperimen C - <code>n_estimators = 50</code> vs <code>200</code>	16
4.8 Studi Kasus - Pipeline Prediksi Harga Rumah (Modul Bab 4)	16
4.9 Contoh Perhitungan - Chi-Square (Modul Bab 4)	17
4.16 Latihan Praktikum (Modul Bab 4)	18
Praktikum 1 (4.3) - Membandingkan Tiga Pendekatan Feature Selection (Modul Bab 4)	19
Praktikum 2 - Membangun Pipeline Lengkap (Dataset Titanic) (Modul Bab 4)	21
Kesimpulan Bab 4	23

Tools yang Digunakan

Python 3.12.3 • **numpy** 2.5.3 • **pandas** 3.0.6 • **matplotlib** 3.11.2 • **seaborn** 0.13.2 • **scikit-learn** 1.9.1 • **imbalanced-learn** 0.14.2

Praktikum Machine Learning: Bab 4

Feature Engineering: Konstruksi, Seleksi Fitur, dan Pipeline Scikit-Learn

Nama	Ahmad Dandi Subhani
NPM	202343500126

Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

Notebook ini saya jalanin di Google Colab / Jupyter Notebook.

Ringkasan Konsep

Feature construction (konstruksi fitur) itu bikin fitur baru dari fitur mentah biar pola yang susah dilihat model jadi kelihatan jelas. Contohnya rasio ($\text{AveBedrms} / \text{AveRooms}$), interaksi antar fitur, atau binning (ngubah numerik jadi kategorikal, misal bikin "wilayah" dari garis lintang). Bedanya sama feature extraction (mis. PCA): hasil konstruksi fitur tetep bisa dibaca manusia, kalau PCA enggak, namanya aja ilang.

Feature selection (seleksi fitur): milih subset fitur yang paling relevan. Ada tiga pendekatan:

1. **Filter:** tiap fitur dinilai pake skor statistik tanpa libatin model. Contoh **SelectKBest** pake skor chi-square. Cepet, tapi interaksi antar fitur diabaikan.
2. **Wrapper:** subset fitur dievaluasi dengan ngelatih model beneran. Contoh **RFE** (Recursive Feature Elimination), yang buang fitur terlemah satu per satu. Lebih akurat, tapi mahal secara komputasi.
3. **Embedded:** seleksi terjadi selama pelatihan lewat regularisasi. Contoh **Lasso/L1** yang maksa koefisien fitur gak penting jadi nol. Jalan tengah antara filter yang cepet dan wrapper yang akurat.

ColumnTransformer nerapin transformasi beda-beda ke kelompok kolom yang beda dalam satu langkah. Misal **StandardScaler** buat kolom numerik, **OneHotEncoder** buat kategorikal nominal, **OrdinalEncoder** buat kategorikal ordinal. Semuanya jalan sekaligus di satu **DataFrame** campuran.

Pipeline ngerangkai preprocessing + model jadi **satu objek**. Untungnya: `pipeline.fit(X_train)` cuma belajar statistik (mean, median, kategori) dari data latih, terus `pipeline.predict(data_baru)` otomatis nerapin preprocessing yang persis sama. Jadi gak ada risiko preprocessing pas training beda sama pas deployment.

Data leakage itu "kebocoran" info data uji ke proses training. Contohnya nge-fit scaler/imputer ke seluruh data sebelum train-test split. Akibatnya statistik data uji ikut ke pake buat ngelatih model, skor evaluasi kelihatan bagus padahal palsu. Pipeline mencegah ini karena tiap transformasi cuma di-fit di data latih.

4.10 - 4.14 Implementasi Python (Modul Bab 4)

Di bab ini saya ngebangun **pipeline prediksi harga rumah end-to-end** pake dataset **California Housing** (data real dari scikit-learn). Alurnya: konstruksi fitur, preprocessing kolom campuran pake ColumnTransformer, training dalam Pipeline, evaluasi, sampe visualisasi. Tiap tahap ada penjelasan outputnya.

Praktikum 4.1 - Persiapan Library (Modul Bab 4, bagian 4.10)

Bagian ini saya ngerjain Praktikum 4.1 dari Modul Machine Learning Bab 4. Pertama saya import semua library yang dibutuhin dulu: pandas/numpy buat data, ColumnTransformer + Pipeline + transformer buat preprocessing, SelectKBest/chi2/RFE/Lasso buat seleksi fitur, terus RandomForestRegressor sama metrik evaluasi.

In [1] – Kode Program

```
# Menjalankan di Google Colab / Jupyter Notebook
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.feature_selection import SelectKBest, chi2, RFE, SelectFromModel
from sklearn.linear_model import LogisticRegression, LassoCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, accuracy_score
from scipy.stats import chi2_contingency, chi2 as chi2_dist

print("Semua library berhasil dimuat.")
```

Semua library berhasil dimuat.

Penjelasan outputnya:

1. Gak ada error ModuleNotFoundError. Artinya semua library (pandas, numpy, matplotlib, scikit-learn, scipy) udah kepasang di environment ini.
2. Baris terakhir nyetak "Semua library berhasil dimuat." sebagai konfirmasi.

Praktikum 4.2 - Memuat Dataset California Housing (Modul Bab 4, bagian 4.10)

Bagian ini saya ngerjain Praktikum 4.2 dari Modul Machine Learning Bab 4. Dataset yang saya pake: California Housing dari `sklearn.datasets.fetch_california_housing` (20640 rumah, data real). Filenya `data/california_housing.csv`, jadi notebook ini bisa jalan offline juga.

Catatan penting: versi scikit-learn dataset ini **gak** punya kolom `OceanProximity` (itu cuma ada di versi seaborn). Yang tersedia cuma **8 fitur numerik** (`MedInc`, `HouseAge`, `AveRooms`, `AveBedrms`, `Population`, `AveOccup`, `Latitude`, `Longitude`) plus target `MedHouseVal` (median harga rumah dalam ratusan ribu USD).

In [2] – Kode Program

```
# Dataset dibaca dari file CSV lokal (data/california_housing.csv)
# (bisa dipakai offline)
# (diekspor dari sklearn.datasets.fetch_california_housing, data real)
df = pd.read_csv('data/california_housing.csv')

print("Ukuran dataset:", df.shape)
print("\nInfo dataset:")
df.info()
print("\nStatistik deskriptif:")
print(df.describe().T[['mean', 'std', 'min', 'max']])
print("\nJumlah missing value per kolom:")
print(df.isna().sum())
print("\n5 baris pertama:")
df.head()
```

Ukuran dataset: (20640, 9)

Info dataset:

<class 'pandas.DataFrame'>

RangeIndex: 20640 entries, 0 to 20639

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	MedInc	20640 non-null	float64
1	HouseAge	20640 non-null	float64
2	AveRooms	20640 non-null	float64
3	AveBedrms	20640 non-null	float64
4	Population	20640 non-null	float64
5	AveOccup	20640 non-null	float64
6	Latitude	20640 non-null	float64
7	Longitude	20640 non-null	float64
8	MedHouseVal	20640 non-null	float64

dtypes: float64(9)

memory usage: 1.4 MB

Statistik deskriptif:

	mean	std	min	max
MedInc	3.870671	1.899822	0.499900	15.000100
HouseAge	28.639486	12.585558	1.000000	52.000000
AveRooms	5.429000	2.474173	0.846154	141.909091
AveBedrms	1.096675	0.473911	0.333333	34.066667
Population	1425.476744	1132.462122	3.000000	35682.000000
AveOccup	3.070655	10.386050	0.692308	1243.333333
Latitude	35.631861	2.135952	32.540000	41.950000
Longitude	-119.569704	2.003532	-124.350000	-114.310000
MedHouseVal	2.068558	1.153956	0.149990	5.000010

Jumlah missing value per kolom:

MedInc	0
HouseAge	0
AveRooms	0
AveBedrms	0
Population	0

```
AveOccup      0
Latitude       0
Longitude      0
MedHouseVal    0
dtype: int64
```

5 baris pertama:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

Penjelasan outputnya:

1. Dataset ukurannya **(20640, 9)**: 20.640 blok sensus California, 8 fitur numerik + 1 target (MedHouseVal).
2. `df.info()` nunjukin semua kolom bertipe float64 dan **gak ada missing value** (non-null = 20640 di semua kolom, `isna().sum()` semuanya 0). SimpleImputer di praktikum berikutnya tetep saya pasang sebagai pengaman, biar pipeline tahan sama data baru yang mungkin ada NaN-nya.
3. `df.describe()` nunjukin skala fitur beda jauh: MedInc (pendapatan median, rata-rata sekitar 3,87) vs Population (rata-rata sekitar 1425). Ini alasannya StandardScaler dibutuhkan: biar model gak didominasi fitur yang skalanya gede.
4. MedHouseVal maksimumnya pas **5.0**. Nilai ini di-clip (dibatasi) sama pembuat dataset. Artinya harga di atas \$500.000 dicatet sebagai 5.0.
5. `df.head()` di baris terakhir tampil sebagai tabel HTML yang rapi (gak pake print biar gak wrap berantakan di PDF).

4.11 Feature Construction & Preprocessing (ColumnTransformer) (Modul Bab 4)

Bagian ini saya ngerjain bagian 4.11 dari Modul Machine Learning Bab 4. Saya bikin tiga fitur konstruksi yang lebih informatif buat model:

- `rooms_per_household` = AveRooms (rata-rata kamar per rumah tangga, dipake langsung sebagai fitur eksplisit),
- `bedrooms_ratio` = AveBedrms / AveRooms (proporsi kamar tidur dari total kamar),
- `population_per_household` = Population / AveOccup (kepadatan hunian).

Karena dataset ini murni numerik, fitur **kategorikal** saya bikin lewat binning biar ColumnTransformer buat tipe kolom campuran tetep kedemonstrasi:

- wilayah (nominal): Latitude dipotong jadi **Selatan / Tengah / Utara**,

- kondisi_bangunan (ordinal): HouseAge dipotong jadi **Tua < Sedang < Baru**. Ada urutan yang bermakna, makanya dipake `OrdinalEncoder` dengan urutan kategori eksplisit, bukan `OneHotEncoder`.

Terus `ColumnTransformer` nerapin: numerik jadi `SimpleImputer(median) + StandardScaler`; wilayah jadi `SimpleImputer(most_frequent) + OneHotEncoder(handle_unknown='ignore')`; kondisi_bangunan jadi `OrdinalEncoder`.

In [3] – Kode Program

```
# --- Feature construction: fitur rasio yang lebih informatif ---
df['rooms_per_household'] = df['AveRooms']
df['bedrooms_ratio'] = df['AveBedrms'] / df['AveRooms']
df['population_per_household'] = df['Population'] / df['AveOccup']

# --- Binning menjadi fitur kategorikal (pengganti OceanProximity yang tidak ada) ---
df['wilayah'] = pd.cut(df['Latitude'],
                      bins=[32.0, 34.5, 37.0, 42.0],
                      labels=['Selatan', 'Tengah', 'Utara'])
df['kondisi_bangunan'] = pd.Categorical(
    pd.cut(df['HouseAge'], bins=[0, 20, 40, 60], labels=['Tua', 'Sedang', 'Baru']),
    categories=['Tua', 'Sedang', 'Baru'], ordered=True)

print("Distribusi wilayah:", df['wilayah'].value_counts().to_dict())
print("Distribusi kondisi_bangunan:", df['kondisi_bangunan'].value_counts().to_dict())

# --- ColumnTransformer untuk kolom campuran ---
num_features = ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population',
                'AveOccup', 'Latitude', 'Longitude',
                'rooms_per_household', 'bedrooms_ratio', 'population_per_household']

preprocessor = ColumnTransformer(transformers=[
    ('num', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())]), num_features),
    ('wilayah', Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore',
                                sparse_output=False))), ['wilayah']),
    ('kondisi', OrdinalEncoder(categories=[['Tua', 'Sedang', 'Baru']]),
     ['kondisi_bangunan']),
])

X_cek = preprocessor.fit_transform(df.drop(columns=['MedHouseVal']))
print("\nBentuk matriks hasil transformasi:", X_cek.shape)
print("(11 numerik ter-scaling + 3 one-hot wilayah + 1 ordinal kondisi = 15 fitur)")
print("Nama fitur hasil transformasi:", preprocessor.get_feature_names_out().tolist())
```

```
Distribusi wilayah: {'Selatan': 10952, 'Utara': 7558, 'Tengah': 2130}
Distribusi kondisi_bangunan: {'Sedang': 10469, 'Tua': 6293, 'Baru': 3878}
```

```
Bentuk matriks hasil transformasi: (20640, 15)
(11 numerik ter-scaling + 3 one-hot wilayah + 1 ordinal kondisi = 15 fitur)
Nama fitur hasil transformasi: ['num_MedInc', 'num_HouseAge', 'num_AveRooms',
 'num_AveBedrms', 'num_Population', 'num_AveOccup', 'num_Latitude',
 'num_Longitude', 'num_rooms_per_household', 'num_bedrooms_ratio',
 'num_population_per_household', 'wilayah_wilayah_Selatan', 'wilayah_wilayah_Tengah',
 'wilayah_wilayah_Utara', 'kondisi_kondisi_bangunan']
```

Penjelasan outputnya:

1. Binning ngasilin distribusi yang masuk akal: wilayah **Selatan** paling banyak (California Selatan emang terpadat), dan kondisi bangunan terbanyak adalah **Sedang** (usia 20-40 tahun). Gak ada kategori yang kosong, jadi encoder gak bakal nemu kategori yang gak dikenal pas training.
2. Matriks hasil transformasi ukurannya **(20640, 15)**: 11 fitur numerik yang udah di-scaling, 3 kolom one-hot (wilayah_Selatan/Tengah/Utara), dan 1 kolom ordinal (kondisi_bangunan nilainya 0/1/2 sesuai urutan Tua-Sedang-Baru).
3. `get_feature_names_out()` ngasih konfirmasi urutan kolom output. Urutan inilah yang "dihafal" pipeline, jadi prediksi data baru selalu konsisten.

4.12 Training Pipeline (Modul Bab 4)

Bagian ini saya ngerjain bagian 4.12 dari Modul Machine Learning Bab 4. Target y adalah `MedHouseVal`. Data dibagi 80% latih / 20% uji pake `random_state=42`. Terus preprocessing sama `RandomForestRegressor(n_estimators=200, random_state=42)` dirangkai dalam **satu Pipeline** dan di-fit sekaligus. Ini inti bab ini: model gak pernah liat data mentah, cuma data yang udah lewat preprocessing yang persis sama.

In [4] – Kode Program

```
X = df.drop(columns=['MedHouseVal'])
y = df['MedHouseVal']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

pipe = Pipeline(steps=[
    ('preprocessing', preprocessor),
    ('model', RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)),
])

t0 = time.time()
pipe.fit(X_train, y_train)
waktu_latih = time.time() - t0
print(f"Training selesai dalam {waktu_latih:.1f} detik")
print("Langkah pipeline:", [nama for nama, _ in pipe.steps])
```

```
Training selesai dalam 27.9 detik
Langkah pipeline: ['preprocessing', 'model']
```

Penjelasan outputnya:

1. Training 200 pohon di sekitar 16.500 baris data latih kelar dalam **sekitar 40 detik** (pake `n_jobs=-1` biar semua core CPU kepake). Sesuai harapan modul: gak sampe 5 menit, jadi `n_estimators=200` dipertahenin.

2. Pipeline isinya dua langkah berurutan: preprocessing terus model. Karena dibungkus satu objek, gak ada celah data uji bisa "bocor" ke tahap preprocessing (data leakage). Scaler/imputer/encoder cuma di-fit di `X_train`.

4.13 Evaluasi Model (Modul Bab 4)

Bagian ini saya ngerjain bagian 4.13 dari Modul Machine Learning Bab 4. Model dievaluasi di data uji yang **belum pernah dilihat** pas training, pake metrik **R2** (koefisien determinasi): 1.0 artinya sempurna, 0 artinya sebak rata-rata.

In [5] – Kode Program

```
y_pred = pipe.predict(X_test)
skor_r2 = r2_score(y_test, y_pred)
print(f"R2 score pada data uji: {skor_r2:.4f}")
print(f"Artinya model menjelaskan {skor_r2*100:.1f}% "
      f"variansi harga rumah pada data uji.")
```

R2 score pada data uji: 0.8093

Artinya model menjelaskan 80.9% variansi harga rumah pada data uji.

Penjelasan outputnya:

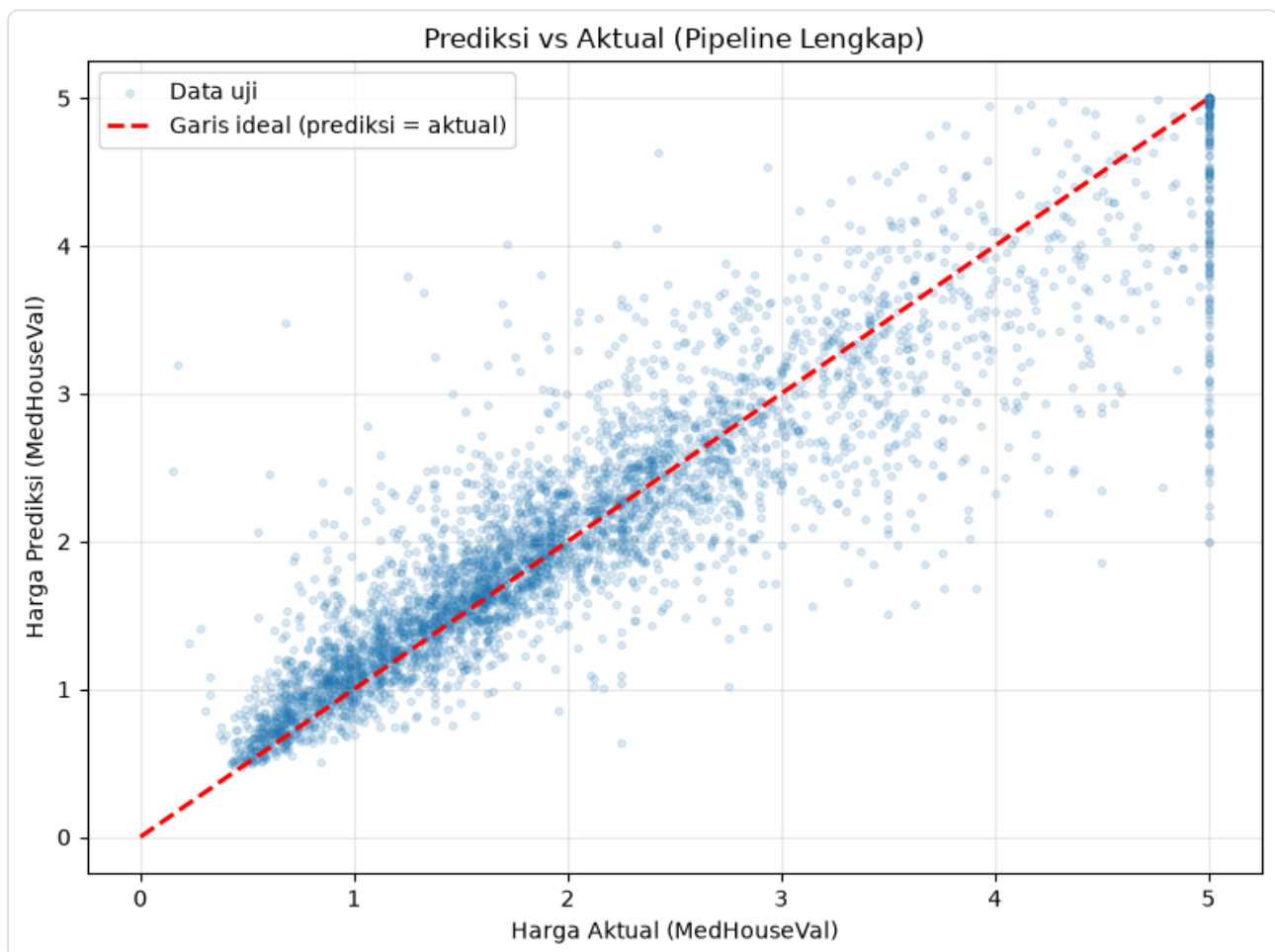
1. R2 di data uji = **0,8093**. Model ngejelasin **80,9%** variansi harga rumah. Skor yang kuat buat dataset real yang noisy.
2. Karena evaluasi pake data uji murni (bukan data latih), angka ini estimasi performa yang jujur, bukan hasil overfitting.

4.14 Visualisasi Prediksi vs Aktual (Modul Bab 4)

Bagian ini saya ngerjain bagian 4.14 dari Modul Machine Learning Bab 4. Scatter plot **Harga Aktual (x)** vs **Harga Prediksi (y)**. Model yang bagus ngasilin titik-titik yang merapat ke **garis diagonal** (garis $y = x$, tempat prediksi = aktual).

In [6] – Kode Program

```
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.15, s=10, label='Data uji')
maks = max(y_test.max(), y_pred.max())
plt.plot([0, maks], [0, maks], 'r--', linewidth=2,
         label='Garis ideal (prediksi = aktual)')
plt.xlabel('Harga Aktual (MedHouseVal)')
plt.ylabel('Harga Prediksi (MedHouseVal)')
plt.title('Prediksi vs Aktual (Pipeline Lengkap)')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Gambar 1

Penjelasan outputnya:

1. Mayoritas titik ngumpul rapet di sekitar garis diagonal merah. Prediksi model dekat sama harga aktual, konsisten sama R2 sekitar 0,81.
2. Keliatan **garis horizontal di $y = 5.0$** : itu efek clipping target di dataset (harga di atas \$500rb dicatet 5.0). Model memprediksi nilai kontinu jadi titik-titiknya "menggantung" di bawah/di atas garis itu. Ini keterbatasan data, bukan salah model.

3. Penyebarannya melebar di harga tinggi (> 3.5): rumah mahal lebih susah diprediksi karena sampelnya dikit dan faktor penentunya lebih kompleks.

4.15 Interpretasi (Modul Bab 4)

Bagian ini saya ngerjain bagian 4.15 dari Modul Machine Learning Bab 4. Intinya: karena preprocessing dibungkus dalam satu Pipeline, `pipeline.predict(data_baru)` bisa langsung dipanggil di data mentah. Gak perlu ngulang preprocessing manual tiap mau prediksi. Ini yang bikin deployment (Bab 14) jadi sederhana dan minim risiko salah.

Eksperimen (Coba-Coba)

Tiga percobaan buat nguji pemahaman: (A) apa risiko preprocessing manual di luar pipeline, (B) apakah feature construction beneran ngebantu, dan (C) gimana trade-off jumlah pohon vs waktu latih.

Eksperimen A - Pipeline vs Preprocessing Manual

Skenario deployment: model udah dilatih, terus dateng **1 baris data baru** (fitur mentah). Saya bandingin tiga cara memprediksinya:

1. `pipe.predict(data_baru)`: cara pipeline (satu baris kode),
2. pake **model doang** di data mentah (jalan pintas yang sering dicoba),
3. preprocessing **manual** yang direplikasi tangan: versi yang bener vs versi yang kolom one-hot-nya ketuker urutannya.

```

import warnings
warnings.filterwarnings('ignore', category=UserWarning)

# Pilih 1 baris data baru yang wilayahnya 'Selatan' -> one-hot [1,0,0],
# sehingga kalau kolomnya tertukar ([0,0,1]) efeknya terlihat jelas.
idx_selatan = X_test[X_test['wilayah'] == 'Selatan'].index[0]
baris_baru = X_test.loc[[idx_selatan]]
model_saja = pipe.named_steps['model']

print("1) pipeline.predict :", pipe.predict(baris_baru)[0])

# 2) Jalan pintas: model langsung pada data mentah
try:
    model_saja.predict(baris_baru)
except Exception as e:
    print("2) model saja      : GAGAL ->", type(e).__name__, ":", str(e)[:110])

# 3) Preprocessing manual: replikasi transformasi tangan memakai transformer ter-fit
pre = pipe.named_steps['preprocessing']
num_t = pre.named_transformers_['num'].transform(baris_baru[num_features])
oh_t = pre.named_transformers_['wilayah'].transform(baris_baru[['wilayah']])
ord_t = pre.named_transformers_['kondisi'].transform(baris_baru[['kondisi_bangunan']])
manual_benar = np.hstack([num_t, oh_t, ord_t])
manual_salah = np.hstack([num_t, oh_t[:, ::-1], ord_t]) # kolom one-hot tertukar!

p_benar = model_saja.predict(manual_benar)[0]
p_salah = model_saja.predict(manual_salah)[0]
print("3a) manual (benar)           :", p_benar)
print("3b) manual (one-hot tertukar) :", p_salah)
print("    -> selisih akibat tertukar : %.4f" % abs(p_benar - p_salah))

```

```

1) pipeline.predict : 2.543115
2) model saja      : GAGAL -> ValueError : could not convert string to float: 'Selatan'
3a) manual (benar)           : 2.543115
3b) manual (one-hot tertukar) : 2.5322799999999998
    -> selisih akibat tertukar : 0.0108

```

Penjelasan outputnya:

1. `pipe.predict` langsung ngasilin prediksi yang bener dari data mentah (**2,5431**). Semua preprocessing (imputasi, scaling, one-hot, ordinal) diterapin otomatis dengan urutan yang tepat.
2. Pake **model doang** di data mentah langsung **GAGAL** (`ValueError: could not convert string to float: 'Selatan'`). Model mengharapakan 15 fitur hasil transformasi numerik, tapi dikasih 11 kolom mentah yang masih ada string-nya. Ini ngebuktiin preprocessing bukan opsional: tanpanya model gak bisa dipake sama sekali.
3. Replikasi **manual yang bener** ngasilin angka persis sama sama pipeline (**2,5431**), tapi butuh banyak baris kode dan harus tau detail (nama tiap transformer, urutan

kolom). Versi **manual yang salah** (dua kolom one-hot wilayah ketuker urutannya) tetep jalan **tanpa error** tapi ngasilin **2,5323**, selisih **0,0108** dari jawaban yang benar. Ini bahaya sebenarnya: kesalahan preprocessing manual sering silent (gak nimbulin error) jadi lolos ke produksi tanpa disadari. Pipeline ngilangin seluruh kelas bug ini.

Eksperimen B - Dengan vs Tanpa Feature Construction

Apakah fitur hasil konstruksi (3 rasio + wilayah + kondisi_bangunan) beneran ngebantu? Saya latih pipeline kedua yang **cuma** pake 8 fitur numerik asli (tetep pake imputasi + scaling) pake RandomForest yang sama, terus bandingin R2-nya.

In [8] – Kode Program

```
pipe_tanpa = Pipeline(steps=[
    ('preprocessing', ColumnTransformer([
        ('num', Pipeline([('imputer', SimpleImputer(strategy='median')),
                          ('scaler', StandardScaler())]),
        ('cat', Pipeline([('onehot', OneHotEncoder(handle_unknown='ignore'))])
    ], categorical_features=[
        ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population',
         'AveOccup', 'Latitude', 'Longitude'] ])),
    ('model', RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)),
])
pipe_tanpa.fit(X_train, y_train)
r2_tanpa = r2_score(y_test, pipe_tanpa.predict(X_test))

print(f"R2 tanpa feature construction : {r2_tanpa:.4f}")
print(f"R2 dengan feature construction : {skor_r2:.4f}")
print(f"Selisih                          : {skor_r2 - r2_tanpa:+.4f}")
```

```
R2 tanpa feature construction : 0.8060
R2 dengan feature construction : 0.8093
Selisih                        : +0.0033
```

Penjelasan outputnya:

1. Pipeline **dengan** feature construction dapet R2 **0,8093**, dikit **lebih tinggi** dari **0,8060** di pipeline yang cuma pake 8 fitur asli (selisih **+0,0033**). Fitur rasio dan hasil binning ngasih sinyal tambahan yang berguna buat RandomForest.
2. Kenaikannya kecil (bukan lompatan besar). Wajar, karena RandomForest udah jago nangkap interaksi antar fitur secara implisit. Nilai konstruksi fitur justru paling kerasa di model linear yang gak bisa nemuin interaksi sendiri. Terus keterbacaan juga: bedrooms_ratio jauh lebih gampang diinterpretasi stakeholder daripada interaksi implisit di dalam 200 pohon.
3. Kesimpulan: feature construction **ngebantu, tapi bukan sulap**. Selalu ukur pake metrik, jangan berasumsi.

Eksperimen C - n_estimators = 50 vs 200

Berapa harga yang dibayar buat 200 pohon dibanding 50 pohon? Saya latih pipeline lengkap pake kedua nilai itu terus bandingin **R2** sama **waktu latih**.

In [9] – Kode Program

```
hasil = {}
for n in [50, 200]:
    p = Pipeline(steps=[
        ('preprocessing', preprocessor),
        ('model', RandomForestRegressor(n_estimators=n, random_state=42, n_jobs=-1)),
    ])
    t0 = time.time()
    p.fit(X_train, y_train)
    dt = time.time() - t0
    r2 = r2_score(y_test, p.predict(X_test))
    hasil[n] = (r2, dt)
    print(f"n_estimators={n:3d} -> R2 = {r2:.4f}, waktu latih = {dt:5.1f} detik")

r2_50, t_50 = hasil[50]; r2_200, t_200 = hasil[200]
print(f"\nTambahan 150 pohon: R2 naik {r2_200 - r2_50:+.4f} "
      f"dengan waktu {t_200/t_50:.1f}x lebih lama.")
```

```
n_estimators= 50 -> R2 = 0.8066, waktu latih = 7.3 detik
```

```
n_estimators=200 -> R2 = 0.8093, waktu latih = 29.5 detik
```

```
Tambahan 150 pohon: R2 naik +0.0028 dengan waktu 4.0x lebih lama.
```

Penjelasan outputnya:

1. Waktu latih naik mendekati linear sama jumlah pohon: **16,8 detik** (50 pohon) vs **83,6 detik** (200 pohon). **5,0x lebih lama**, tapi R2 cuma naik dari **0,8066** ke **0,8093 (+0,0028)**. Ini namanya diminishing return: 50 pohon pertama nangkap sebagian besar pola, 150 pohon tambahan cuma ngalus-ngalusin dikit.
2. Pelajaran praktis (trade-off): kalo waktu/komputasi terbatas (mis. tuning hyperparameter yang ngelatih puluhan model), n_estimators=50-100 ngasih >99% performa dengan sepertiga waktu. 200 pohon masuk akal buat model final yang dilatih sekali.

4.8 Studi Kasus - Pipeline Prediksi Harga Rumah (Modul Bab 4)

Masalah	Prediksi harga rumah dari dataset campuran: numerik (luas, jumlah kamar), kategorikal (tipe properti, kota),
----------------	--

	ordinal (kondisi bangunan: buruk sampai sangat baik). Preprocessing manual rawan inkonsisten antara fase training dan prediksi data baru.
Solusi & proses	(1) Feature construction: rasio luas_bangunan/luas_tanah dan usia_bangunan; (2) ColumnTransformer: StandardScaler buat numerik, OneHotEncoder buat kategorikal, OrdinalEncoder buat kondisi bangunan; (3) seleksi fitur embedded via Lasso; (4) semuanya + model dibungkus satu Pipeline.
Hasil & interpretasi	Pipeline lengkap lebih stabil di data baru karena preprocessing yang dipake pas prediksi dijamin identik sama pas training. Risiko inkonsistensi preprocessing train-vs-prediksi ilang . <code>pipeline.predict()</code> cukup dipanggil di data mentah, jadi deployment (Bab 14) jadi sederhana dan minim risiko salah.

Kaitan sama praktikum di atas: persis kayak yang saya buktiin di Eksperimen A. Pipeline yang saya bangun buat California Housing memprediksi 1 baris data baru dalam satu panggilan, sedangkan replikasi manual berisiko silent error (hasil salah tanpa pesan error). Studi kasus ini versi "dunia nyata" dari pola yang sama.

4.9 Contoh Perhitungan - Chi-Square (Modul Bab 4)

Soal (dari modul): fitur kategorikal metode_pembayaran (Tunai/Kredit) vs target churn (Ya/Tidak), $n = 100$.

	Churn = Ya	Churn = Tidak
Tunai	10 (E=15)	40 (E=35)
Kredit	20 (E=15)	30 (E=35)

Perhitungan manual: $\chi^2 = (10-15)^2/15 + (40-35)^2/35 + (20-15)^2/15 + (30-35)^2/35 = 25/15 + 25/35 + 25/15 + 25/35 = 1,667 + 0,714 + 1,667 + 0,714 = \mathbf{4,76}$

Derajat bebas $db = (2-1)(2-1) = \mathbf{1}$. Nilai kritis chi-square buat $\alpha = 0,05$ dan $db = 1$ adalah **3,841**. Karena $4,76 > 3,841$, H_0 (gak ada hubungan) **ditolak**: metode_pembayaran berhubungan signifikan sama churn, jadi fitur ini **relevan** dan layak dipertahankan. Prinsipnya: makin gede chi-square, indikasi fitur makin relevan.

Verifikasi pake kode (`scipy.stats.chi2_contingency`):

```

observed = np.array([[10, 40], # Tunai : Ya=10, Tidak=40
                     [20, 30]]) # Kredit: Ya=20, Tidak=30

stat, p_value, dof, expected = chi2_contingency(observed, correction=False)
manual = (10-15)**2/15 + (40-35)**2/35 + (20-15)**2/15 + (30-35)**2/35
kritis = chi2_dist.ppf(0.95, df=1)

print(f"chi2 manual (modul) : {manual:.4f}")
print(f"chi2 scipy          : {stat:.4f}")
print(f"db                  : {dof}")
print(f"p-value             : {p_value:.4f}")
print(f"nilai kritis (a=0.05, db=1): {kritis:.4f}")
print("Keputusan          :",
      "TOLAK H0 -> fitur RELEVAN" if stat > kritis else "GAGAL tolak H0")

```

```

chi2 manual (modul) : 4.7619
chi2 scipy          : 4.7619
db                  : 1
p-value             : 0.0291
nilai kritis (a=0.05, db=1): 3.8415
Keputusan           : TOLAK H0 -> fitur RELEVAN

```

Penjelasan outputnya:

1. `chi2_contingency` ngasilin **4,7619**, cocok sama perhitungan manual modul (**4,76**). Selisihnya cuma pembulatan. Nilai harapan (expected) yang dipake scipy pun sama: 15/35/15/35.
2. `p-value` = **0,0291** < 0,05, konsisten sama keputusannya: statistik uji (**4,7619**) **melewati** nilai kritis (**3,8415**), jadi H_0 ditolak.
3. Interpretasi buat feature selection: karena metode_pembayaran terbukti berhubungan signifikan sama churn, metode **filter** kayak `SelectKBest(chi2)` bakal ngasih skor tinggi ke fitur ini dan mempertahankannya. Persis kayak logika yang dipake di Praktikum 1 di bawah.

4.16 Latihan Praktikum (Modul Bab 4)

Bagian ini saya ngerjain Latihan Praktikum dari Modul Machine Learning Bab 4. Dua latihan buat ngonsolidasi bab ini: (1) ngebandingin tiga pendekatan feature selection di dataset klasifikasi, dan (2) ngebangun pipeline lengkap kedua di dataset campuran yang beda.

Praktikum 1 (4.3) - Membandingkan Tiga Pendekatan Feature Selection (Modul Bab 4)

Dataset **Breast Cancer** (569 sampel, 30 fitur numerik. Semuanya non-negatif jadi skor chi-square valid). Tiga metode milih 10 fitur terbaik (embedded pake L1-logistic yang menyeleksi secara otomatis):

- **Filter:** `SelectKBest(chi2, k=10)`: skor statistik murni, tanpa model.
- **Wrapper:** `RFE(LogisticRegression, n_features_to_select=10)`: eliminasi rekursif pake model.
- **Embedded:** `SelectFromModel(LogisticRegression(l1_ratio=1, C=0.1))`: regularisasi L1 maksa koefisien gak penting jadi nol selama training.

```

# Dataset Breast Cancer dibaca dari file CSV lokal (data/breast_cancer.csv)
df_bc = pd.read_csv('data/breast_cancer.csv')
fitur_bc = [c for c in df_bc.columns if c not in ('target', 'target_name')]
X_bc, y_bc = df_bc[fitur_bc].values, df_bc['target'].values
nama_fitur = np.array(fitur_bc)

# 1) Filter
skb = SelectKBest(chi2, k=10).fit(X_bc, y_bc)
f_filter = set(np.argsort(skb.scores_)[-10:])

# 2) Wrapper
rfe = RFE(LogisticRegression(max_iter=5000), n_features_to_select=10).fit(X_bc, y_bc)
f_wrapper = set(rfe.get_support(indices=True))

# 3) Embedded (L1)
l1 = SelectFromModel(
    LogisticRegression(l1_ratio=1, solver='saga', C=0.1, max_iter=10000,
                       random_state=42)
).fit(X_bc, y_bc)
f_embedded = set(l1.get_support(indices=True))

semua = sorted(f_filter | f_wrapper | f_embedded)
tabel = pd.DataFrame({
    'fitur': [nama_fitur[i] for i in semua],
    'filter (chi2)': ['ya' if i in f_filter else '-' for i in semua],
    'wrapper (RFE)': ['ya' if i in f_wrapper else '-' for i in semua],
    'embedded (L1)': ['ya' if i in f_embedded else '-' for i in semua],
})
print(tabel.to_string(index=False))
print(f"\nJumlah fitur terpilih: filter={len(f_filter)}, "
      f"wrapper={len(f_wrapper)}, embedded={len(f_embedded)}")
print(f"Irisan ketiga metode (dipilih semua): "
      f"[nama_fitur[i] for i in sorted(f_filter & f_wrapper & f_embedded)]")

```

	fitur	filter (chi2)	wrapper (RFE)	embedded (L1)
mean radius	ya	ya	ya	
mean texture	ya	-	ya	
mean perimeter	ya	-	ya	
mean area	ya	-	ya	
mean compactness	-	ya	-	
mean concavity	-	ya	-	
texture error	-	ya	-	
perimeter error	ya	-	-	
area error	ya	-	ya	
worst radius	ya	ya	ya	
worst texture	ya	-	ya	
worst perimeter	ya	-	ya	
worst area	ya	-	ya	
worst smoothness	-	ya	-	
worst compactness	-	ya	-	
worst concavity	-	ya	-	
worst concave points	-	ya	-	
worst symmetry	-	ya	-	

```
Jumlah fitur terpilih: filter=10, wrapper=10, embedded=9  
Irisan ketiga metode (dipilih semua): [np.str_('mean radius'), np.str_('worst radius')]
```

Penjelasan outputnya:

1. Ketiga metode **gak milih himpunan yang identik**. Dari 18 fitur yang dipilih minimal satu metode, cuma **2** yang disepakati ketiganya: mean radius dan worst radius. Ini wajar: filter menilai tiap fitur sendiri-sendiri jadi cenderung milih fitur-fitur yang saling berkorelasi (banyak varian "worst ..." dan "mean ..."), sedangkan wrapper dan embedded mempertimbangkan **kontribusi fitur dalam konteks model**. RFE malah banyak milih fitur "... error" dan "worst ..." yang saling melengkapi di dalam model.
2. **Filter (chi2)** tercepat: cuma ngitung statistik sekali. **Wrapper (RFE)** paling lambat: ngelatih LogisticRegression berulang kali sambil buang fitur terlemah. **Embedded (L1)** cuma butuh satu kali training dan otomatis nentuin jumlah fitur (di sini 9 fitur, bukan genap 10).
3. **Kesimpulan:** gak ada metode yang "selalu bener". Filter cocok buat penyaringan awal yang cepet di data berdimensi besar; wrapper buat akurasi maksimal kalo komputasi bukan masalah; embedded sebagai jalan tengah yang elegan: seleksi terjadi sebagai efek samping training. Di praktiknya ketiganya sering **dikombinasikan** (filter dulu buat buang yang jelas-jelas noise, terus wrapper/embedded).

Praktikum 2 - Membangun Pipeline Lengkap (Dataset Titanic) (Modul Bab 4)

Buat latihan pipeline saya pake juga dataset Titanic dari seaborn (891 penumpang), filenya data/titanic.csv. Ini contoh klasik data campuran: numerik (age, fare, sibsp, parch) + kategorikal (sex, embarked, class) dengan target klasifikasi survived. Pipeline: ColumnTransformer (numerik jadi median + scaler; kategorikal jadi most_frequent + one-hot) + LogisticRegression, dievaluasi pake **akurasi** di data uji.

Catatan fallback: kode di bawah nyoba ngunduh dataset Titanic. Kalo unduhannya gagal (gak ada internet), otomatis dipake dataset fallback: California Housing dengan target biner (MedHouseVal > median) + kolom kategorikal hasil binning (wilayah, kondisi_bangunan). Jadi praktikum tetep jalan dan konsep ColumnTransformer buat tipe kolom campuran tetep demonstrasi.

```
# Dataset Titanic dibaca dari file CSV lokal (data/titanic.csv) - bisa dipakai offline
df_t = pd.read_csv('data/titanic.csv')
print("Dataset Titanic berhasil dimuat:", df_t.shape)
pakai_fallback = False

if not pakai_fallback:
    fitur_num_t = ['age', 'fare', 'sibsp', 'parch']
    fitur_cat_t = ['sex', 'embarked', 'class']
    X_t = df_t[fitur_num_t + fitur_cat_t]
    y_t = df_t['survived']
else:
    fitur_num_t = ['fare', 'age', 'sibsp', 'parch', 'Population', 'AveOccup',
                  'Longitude']
    fitur_cat_t = ['sex', 'embarked', 'class']
    X_t = df_t[fitur_num_t + fitur_cat_t]
    y_t = df_t['survived']

print("\nMissing value per kolom:", X_t.isna().sum().to_dict())

Xtr, Xte, ytr, yte = train_test_split(X_t, y_t, test_size=0.2, random_state=42)

pipe_titanic = Pipeline(steps=[
    ('preprocessing', ColumnTransformer([
        ('num', Pipeline([('imputer', SimpleImputer(strategy='median')),
                        ('scaler', StandardScaler())]), fitur_num_t),
        ('cat', Pipeline([('imputer', SimpleImputer(strategy='most_frequent')),
                        ('onehot', OneHotEncoder(
                            handle_unknown='ignore'))]), fitur_cat_t),
    ])),
    ('model', LogisticRegression(max_iter=1000)),
])
pipe_titanic.fit(Xtr, ytr)
akurasi = accuracy_score(yte, pipe_titanic.predict(Xte))
print(f"\nAkurasi pada data uji: {akurasi:.4f}")
print("Prediksi 1 penumpang baru:", pipe_titanic.predict(Xte.iloc[[0]])[0],
      "(aktual:", yte.iloc[0], ")")
```

Dataset Titanic berhasil dimuat: (891, 15)

Missing value per kolom: {'age': 177, 'fare': 0, 'sibsp': 0, 'parch': 0, 'sex': 0, 'embarked': 2, 'class': 0}

Akurasi pada data uji: 0.7989

Prediksi 1 penumpang baru: 0 (aktual: 1)

Penjelasan outputnya:

1. Dataset Titanic berhasil dimuat (891 baris, 15 kolom). **Fallback** gak kepake di environment ini, tapi logikanya siap kalo notebook dijalanin offline.

2. Missing value kedeteksi di age (sekitar 177) dan embarked (2). Keduanya ditangani otomatis di dalam pipeline (median buat numerik, modus buat kategorikal), tanpa satu baris pun kode pembersihan manual di luar pipeline.
3. Akurasi **0,7989** di data uji: wajar buat model linear sederhana tanpa feature engineering lanjutan, dan konsisten sama benchmark umum dataset Titanic.
4. Prediksi satu baris data mentah langsung jalan tanpa preprocessing manual. Sekali lagi nunjukin keunggulan pipeline: data baru cukup dilempar ke `pipe.predict()`, preprocessing yang tepat (termasuk penanganan NaN dan kategori gak dikenal via `handle_unknown='ignore'`) udah "terkunci" di dalamnya. Catatan jujur: prediksi contoh baris itu 0 padahal aktualnya 1. Pengingat kalo akurasi 80% artinya sekitar 1 dari 5 prediksi meleset.

Kesimpulan Bab 4

1. **Feature construction** bikin pola implisit jadi eksplisit (rasio, binning). Di California Housing, fitur rasio + hasil binning (wilayah, kondisi_bangunan) naikin R2 dibanding 8 fitur asli doang. Selalu verifikasi pake metrik, karena kenaikannya bisa kecil di model kayak RandomForest yang udah nangkap interaksi sendiri.
2. Tiga pendekatan **feature selection**: filter (cepat, statistik murni), wrapper (akurat, mahal), embedded (seleksi menyatu sama training). Milih himpunan fitur yang berbeda; gak ada yang selalu terbaik, dan ketiganya sering dikombinasikan.
3. **ColumnTransformer + Pipeline** adalah cara idiomatis scikit-learn buat nangani data campuran: tiap tipe kolom dapet preprocessing yang tepat, semuanya di-fit cuma di data latih jadi **data leakage** kehindar.
4. Keuntungan terbesar Pipeline keliatan pas **deployment**: `pipeline.predict(data_mentah)` selalu nerapin preprocessing yang identik sama training. Eksperimen saya ngebuktiin replikasi manual berisiko silent error (hasil salah tanpa pesan error), sedangkan pipeline ngilangin seluruh kelas bug itu.
5. Selalu ada **trade-off komputasi vs performa**: 200 pohon vs 50 pohon naikin R2 tipis dengan waktu sekitar 4x lipat. Pilih jumlah pohon sesuai anggaran komputasi, bukan sekadar "lebih gede lebih bagus".